

Integrity Constraints: Semantics and Applications

P. Godfrey^{1,5}, J. Grant³, J. Gryz⁴, & J. Minker^{1,2}
godfrey@arl.mil, jgrant@towson.edu, jarek@cs.yorku.ca, minker@cs.umd.edu

¹Department of Computer Science

²and Institute for Advanced Computer Studies
University of Maryland at College Park
College Park, Maryland, U.S.A.

³Department of Computer and Information Sciences
Towson University
Towson, Maryland, U.S.A.

⁴Department of Computer Science
York University
Toronto, Ontario, Canada

⁵U.S. Army Research Laboratory
Adelphi, Maryland, U.S.A.

December 1997

Contents

1	Introduction	3
2	Background	4
3	Semantics of Integrity Constraints	7
3.1	Examples of What Integrity Constraints can Express	8
3.2	Model Semantics	9
3.3	Extensions to the Basic Model	12
4	Reasoning with Integrity Constraints	12
4.1	Eliminating Integrity Constraints	13
4.2	Model Elimination	14
4.3	Residue Method	15
5	Applications of Integrity Constraints	16
5.1	Semantic Query Optimization	16
5.2	Cooperative Answering	18
5.3	Combining Databases and Resolving Inconsistencies	20
5.4	View Updates	22
5.5	Additional Applications	23
6	Conclusion and Future Directions	25

1 Introduction

Databases contain knowledge as well as data. The database’s schema—how the data is organized—is knowledge, which yields constraints on the form the data must take. The relationships between data that must hold, such as functional and inclusion dependencies, are knowledge. General rules about the world or domain, to which the database’s data must always conform, are knowledge as well. Such knowledge defines the semantics of the database. It is beneficial for a database to store explicitly its knowledge, in addition to its data. This has long been recognized in relational databases. Some of the database’s knowledge is captured and stored via *integrity constraints*, statements about what are the legal states and transitions of the database. An integrity constraint is a closed query that must always be *true* after a database update. Integrity constraints (ICs) were introduced to prevent the entering of incorrect data into the database and to check the integrity of the database.

Integrity constraints actually have much wider applicability. In addition to integrity checking, these include query optimization via semantics, cooperative query answering, combining databases in a semantically consistent manner, and view updating. It is commonly held that integrity constraints are an adequate and suitable knowledge representation in databases.¹ Thus the types of knowledge that should be kept by databases can, and should, be written as ICs. By having a standard, uniform representation for the database’s knowledge, the various applications that rely on the database’s semantics can all employ the same representation.

In this chapter, we consider logic databases (also called deductive databases).² Logic databases employ the *logic model*, a subset of the first-order predicate calculus, to describe the database and queries [1, 88]. Records are represented as logical facts. Rules in logic databases allow implicit facts to be derived, via logical deduction. (Views play such a role in relational databases.) The logic model can be extended to allow formulas as integrity constraints. The advantage of taking a logical approach to databases is that data, rules, queries, *and* integrity constraints can be all handled in a common framework, and formal techniques rather than *ad hoc* approaches can then be employed for all database applications.

Consider the following example constraints.

$$\begin{aligned}
 P_1 = P_2 &\Leftarrow \text{employee}(\text{name}: N, \text{address}: P_1), & 1) \\
 &\quad \text{employee}(\text{name}: N, \text{address}: P_2). \\
 &\Leftarrow \text{employee}(\text{dname}: D), \text{not department}(\text{name}: D). & 2) \\
 &\Leftarrow \text{flight}(\text{from}: 'DCA', \text{departs}: \text{Time}), \text{Time} > 22:00. & 3)
 \end{aligned}$$

The ICs are written in an *attribute extended* DATALOG and DATALOG⁺. The basic DATALOG notation is introduced in [21], and the attribute extension, *field notation*, is introduced in Section 3. The first integrity constraint expresses the functional dependency that any two facts (tuples) that agree on *name* must also agree on *address*. (That is, *address* is functionally dependent on *name* in relation *employee*.) The second integrity constraint expresses the inclusion dependency (perhaps also a foreign key dependency) that any value for *dname* in relation *employee* must also appear in the relation *department*. This constraint can be read as “if there is an employee known to be in a department *D*, but there is *no* department *D* known in the *department* relation, then there is an integrity violation.” The third integrity constraint is a general “world” constraint that states that all flights from National Airport (*DCA*) leave before 10:00pm. (That is, it is a violation for any flight to leave after 10:00pm.) See [24] for numerous practical examples of ICs.

Integrity constraints exist in practically all database applications today. Note that the integrity constraints above are expressed declaratively, and in the same logical language with which we express facts and rules. By

¹ There is much work in knowledge representation, and one might argue that another formalism is more suitable for capturing “knowledge” in databases. However, ICs were designed for this purpose, and have proven to be suitable and adequate for the task. (See [52].) We limit our focus to integrity constraints in this chapter, and we do not compare them with other possible knowledge representations here.

² All the results we present, and our considerations, for logic databases apply to relational databases as well. The logic model subsumes the relational model.

having integrity constraints explicitly represented, they are available to all applications alike. A declarative expression of ICs is important, because non-declarative specifications are difficult to express and maintain. Indeed, this same lesson led to the advent of relational databases themselves: a *declarative* specification of facts and rules (views) is easier to express and maintain. Still today, many IC requirements are ensured using application programs. However, the trend is away from this and towards explicit, declarative representation of ICs. Then ICs are not hidden from new applications, and the database system itself can ensure consistency with the ICs without having to rely on application programs to do so. (The SQL-2 standard defines a general class of ICs [71], which subsumes all examples we have in this paper.) Finally, by having ICs expressed in logic, one can use deduction and logic as one's basic and natural tools for handling, and reasoning over, database specifications (facts, rules, queries, and ICs). This applies for relational databases, with or without rules, and can be applied to object-oriented and object-relational databases as well.

There is a broad body of work on logic and relational databases, and a general consensus on what databases (facts and rules) and queries *mean*. However, there is less work on the meaning of integrity constraints, and certainly no consensus. What is meant by an IC can differ widely. For instance, one may define that ICs must be *consistent* with the database, or define that they must be provable statements, deducible from the database. (This distinction is discussed in more detail in Section 3.2.) Another view is that ICs really represent meta-knowledge—knowledge *about* the database itself—and should, perhaps, be written in a non-classical logic. The general situation becomes more complex when we permit databases to contain indefinite or disjunctive information or to use negation, as with IC 2 above. Subtle but profound differences in meaning can arise due to different interpretations of ICs. In many systems, the semantics for ICs is never made clear; at times, one interpretation seems intended, while at other times, another interpretation is evident. This ambiguity is dangerous, and could allow a database to become corrupt in unanticipated ways.

In this chapter, we introduce integrity constraints in a logical framework and overview the various work that has been done on ICs in this context. Section 2 presents the basic definitions for databases and integrity constraints for the chapter. We consider various semantics of ICs, discuss how ICs are used to represent the knowledge of the database and look at variations on the basic model (Section 3), review techniques for reasoning with ICs (Section 4), and present a number of important applications that employ ICs (Section 5). We conclude with a discussion of current and future work (Section 6). There are important aspects of ICs we do not cover in this chapter. In particular, we do not discuss integrity constraint checking; that is, how best to check the consistency of the database with respect to ICs. We recommend [53] as a survey of this work.

2 Background

This section contains a summary of the background and notation used in this chapter. Additional details concerning logic are found in [21]. We use the language and terminology of logic databases ([24], [67], and [68]) as presented in [21]. Logic databases express data, rules (views), and queries in first-order logic ([7] and [67]). We show that integrity constraints can likewise be expressed in first-order logic.³

We use standard syntax for first-order logic, with the usual symbols for variables, connectives, quantifiers, punctuation, equality, constants, and predicates. The notions of term, formula, and sentence (a formula with no free variables) are defined in the usual way. We further restrict formulas to be function-free. Any formula may be considered as a *query* (although, below, we restrict the form of standard queries). A formula is called *ground* if it contains no variables.

A *substitution* is a set of *substitution pairs*, for example, $\{X/a, Y/b\}$, such that every element of a substitution pair is a variable or constant (or, more generally, a term), and such that the collection of left-hand sides of the substitution pairs— X and Y in our example—constitute a set. A substitution *applied* to a formula is a

³Some proposals for integrity constraints express them in a higher-order logic, while keeping the database—the facts, rules, and, sometimes, queries—in first-order. We briefly review some of these proposals in Section 3.3.

rewrite of the formula by replacing any occurrence in the formula of a left-hand element from the substitution by its right-hand counterpart, in parallel. Let the formula $\mathcal{F}$ be $p(X, Y)$ and the substitution θ be again $\{X/a, Y/b\}$. The substitution θ applied to formula $\mathcal{F}$, written as $\mathcal{F}\theta$, is the formula $p(a, b)$. A *ground substitution of a formula* results in a ground formula. When the right-hand sides of the substitution pairs of a substitution θ are distinct and do not appear in the formula $\mathcal{F}$, the *inverse* of the substitution, denoted as θ^{-1} , exists. The inverse is defined as the original substitution with the left-hand and right-hand elements of each of its substitution pairs exchanged. In our example, θ^{-1} is $\{a/X, b/Y\}$.

An *answer* to a query is a ground substitution of the query formula such that the resulting ground formula is *true* with respect to the database; that is, the grounded query formula is *logically entailed* by the database.

An important class of sentence is the *clause*. A clause has the general form:

$$\forall. A_1 \vee \dots \vee A_k \vee \neg A_{k+1} \vee \dots \vee \neg A_n$$

in which each A_i is an atomic formula, for $i \in \{1, \dots, n\}$, and in which the variables are understood to be universally quantified (denoted by ‘ $\forall$ ’). Any clause can be written in a logically equivalent form as an implication.

$$\forall. A_1 \vee \dots \vee A_k \leftarrow A_{k+1} \wedge \dots \wedge A_n.$$

This is often written in further shorthand as

$$A_1, \dots, A_k \leftarrow A_{k+1}, \dots, A_n.$$

in which disjunction is assumed on the left-hand side of the implication arrow, conjunction on the right-hand side, and the universal quantification is understood. A clause in this form is also called a *rule*. The collection of atoms on the left-hand side ($A_1, \dots, A_k$) is called the *head* of the rule, and the collection of atoms on the right-hand side ($A_{k+1}, \dots, A_n$) the *body*. When $k = n$, the body is empty, and when $k = 0$, the head is empty. A *Horn* rule has at most one atom in the head: $k \leq 1$. A *definite* clause has exactly one atom in the head: $k = 1$. A rule is *range-restricted* iff every variable that appears in the head also appears in the body.

A rule with an empty head is generally considered to be a *query*. A ground rule with an empty body is called a *fact*. Definite rules have a clear procedural interpretation. Consider

$$A \leftarrow B_1, \dots, B_n.$$

We call this rule a rule *for* A . The above rule can be interpreted to say that A is shown (or *proven*) whenever all the B_i ’s are shown (*proven*).⁴ Rules are essentially *views*, in the parlance of relational databases. Logically, rules are more expressive than views in relational databases though, when recursion is permitted.⁵ A fact then, having no conditions in the body of its “rule”, is simply interpreted as *true*.

A database may then be defined as a collection of rules and facts. When all the rules and facts are definite (that is, the rules and facts have at most a single atom in the heads of the clauses that define them), the database is called *definite*. It is called *disjunctive* (or *indefinite*) otherwise. We call the language in which the database is written with definite clauses as defined above DATALOG [88]. Recall that terms in clauses are function-free, as noted above, hence DATALOG has terms that are function-free. When disjunctive clauses are permitted for rules or facts (and whose terms are function-free), we call the language Disjunctive DATALOG [28, 68]. Throughout the paper we use the following as synonymous: deductive database, definite deductive database and DATALOG database. Likewise, we treat the following synonymously: disjunctive deductive database and disjunctive DATALOG database. A database **DB** often is defined as consisting of

⁴The interpretation for disjunctive rules is less apparent. Essentially, a disjunctive rule states that at least *one* of the atoms in the rule’s head is *proven* whenever all the atoms in the body are.

⁵The SQL-3 standard extends SQL to support recursion [71], however. So once SQL-3 becomes the standard, this difference in expressiveness will go away, since any relational database that supports SQL-3 will, in fact, be a deductive database system.

two parts:

- the extensional database, $\mathbf{E}_{\mathbf{DB}}$, and
- the intensional database, $\mathbf{I}_{\mathbf{DB}}$.

The $\mathbf{E}_{\mathbf{DB}}$ is the database's collection of *facts*. The $\mathbf{I}_{\mathbf{DB}}$ is the database's collection of rules. (We soon redefine databases to have a third component, the set of the database's integrity constraints.)

Conventionally, negative data is not represented explicitly in a logic database. There are several standard approaches to allow negative data to be inferred. The *closed world assumption* (*CWA*) is a default rule for the inference of negative facts [82]. For any ground atom A , the negation of A is accepted as *true* if A is not provable from the database. The set of all negated atoms inferable in this way is written as $\mathbf{CWA}(\mathbf{DB})$. Another approach to negation is the *Clark completion* of a database [20]. This formalizes the concept that the set of tuples *true* for a predicate is precisely the set that can be proven to be *true* via the facts and rules. In brief, this is accomplished by adding a formula to the database for each predicate (to correspond with the collection of rules for that predicate), to supply the logical *only if* half of the definition of the predicate. Certain negated facts are then *deducible* from the *completed* database, the database with these “only if” formulas added.

So far, we have assumed that the body of a database rule (clause) contains only positive atoms. However, it is useful sometimes to define database rules that allow negated atoms in the body of a rule. We need negation in logic databases if we want to subsume the relational algebra, which includes set difference. We can extend deductive databases with negation. A rule which has a negated atom in its body is called a *normal* rule, and deductive databases that have normal rules are called *normal* databases. We call DATALOG that has been extended with default negation DATALOG⁷. For example, the normal rule

$$p(X) \leftarrow \mathbf{not} \ q(X).$$

is interpreted, in general, to mean that, for any constant a , if $q(a)$ is *not true* (or cannot be *proven* to be *true*), then $p(a)$ is *true*.

We write this negation with ‘**not**’ rather than with the symbol for logical negation, ‘ $\neg$ ’, and refer to it as *default negation*. This is because most semantics that have been defined for normal databases interpret the use of default negation differently from one another and from logical negation. There are a number of semantics that have been defined for normal databases, and no one semantics is universally accepted. Also, since the notion of default negation is generally based on provability, not logical truth, such default negation is beyond first-order logic.⁶

Thus it is not equivalent to exchange the rules in the $\mathbf{I}_{\mathbf{DB}}$ that use default negations with seemingly equivalent disjunctive rules, in which the negated atoms in the body have been moved to the head. Consider the following example.

$\mathbf{DB}_1$	$\mathbf{DB}_2$
$\frac{p \leftarrow q, \mathbf{not} \ r.}{q.}$	$\frac{p \vee r \leftarrow q.}{q.}$

Let $\mathbf{DB}_1$ consist of the single rule for p and the single fact q , and $\mathbf{DB}_2$ of the single disjunctive rule and the single fact q . If the rule in $\mathbf{DB}_1$ were written with logical negation instead of default negation, $\mathbf{DB}_1$ and $\mathbf{DB}_2$ would be logically equivalent. However, in $\mathbf{DB}_1$, we should be able to infer p , because the fact q can be inferred (it is a fact), and the fact r *cannot* be inferred (thus, **not** r can be assumed *true* by default). In $\mathbf{DB}_2$, p cannot be inferred. Only the weaker, disjunctive fact $p \vee r$ can be inferred. Note that default negation results in non-monotonicity. If we were to *add* the fact r to $\mathbf{DB}_1$ above, we would no longer be able

⁶We still speak in terms of first-order logic even for normal databases, as most of the first-order framework of deductive databases remains applicable.

to infer p .

The intuition behind the use of default negation becomes confused when it is combined with recursion. One solution to this confusion is simply not to allow recursive definitions through negation. A canonical example of recursion through negation is

$$\begin{aligned} p(X) &\leftarrow \text{not } q(X). \\ q(X) &\leftarrow \text{not } p(X). \end{aligned}$$

The restriction not to allow recursion through negation leads to what are called *stratified* databases, and such databases have a unique standard model called the *perfect model* of the database. (Stratified databases are defined in [21]. We consider *model semantics* more in Section 3.2 in the context of integrity constraints.) In some cases, a non-stratified database may also have a unique standard model. Some of these cases may be captured by the concept of *stable* database. Two important model semantics for normal databases, and normal logic programs, are the *well-founded semantics* [41] and the *stable model semantics* or the semantically equivalent *well-supported model semantics* [29, 42]. [74] provides a retrospective on work in semantics for logic programs and deductive databases.

People have also considered logics other than first-order logic, besides the addition of default negation, for databases. We consider some extensions to the basic model in Section 3.3. Extensions include modal logics and temporal logics. Such logics extend first order logic with additional operators. Modal logics deal with modalities, such as *possibility* and *belief*. They can be used to distinguish between what is in the database, and *knowledge*, or *beliefs* the database has about what it “knows”. Some of these approaches are discussed in [89]. Temporal logics deal with concepts of time, such as past and future, and the notion of change. They can be used to model transactions in databases. These are discussed in [19].

3 Semantics of Integrity Constraints

We now redefine a deductive database to have three components:⁷

- the extensional database, $\mathbf{E}_{\mathbf{DB}}$,
- the intensional database, $\mathbf{I}_{\mathbf{DB}}$, and
- the integrity constraints, $\mathbf{IC}$.

The formulas in $\mathbf{IC}$ are meant as knowledge about the world, or the domain of the database. They are not intended to generate data, as do the rules in $\mathbf{I}_{\mathbf{DB}}$, nor do they represent specific data, as do the facts in $\mathbf{E}_{\mathbf{DB}}$. Their use is to verify that the data (both *intensional*—deduced by the rules—and *extensional*—facts in $\mathbf{E}_{\mathbf{DB}}$) in the database is consistent with the general model of the world, as represented by the integrity constraints in $\mathbf{IC}$.

Integrity constraints (ICs) in relational databases were introduced for this very purpose: they are used to validate (or invalidate) changes to the database (transactions) to ensure that the new state of the database remains consistent with the world view. However, the world knowledge that the ICs encode has been found to be useful for a wide array of database tasks. We discuss a number of these applications in Section 5.

While the standard view has been that ICs represent knowledge about the world, this position is debatable. In Section 3.3, we briefly consider the view that ICs represent knowledge about the database *itself*. In any case, it is recognized that ICs represent *knowledge*, general characterizations of how things are (and, hence, are a type of knowledge representation), whereas databases themselves— $\mathbf{E}_{\mathbf{DB}}$ and $\mathbf{I}_{\mathbf{DB}}$, which, together, we call the *state* of the database—represent *data*, specific facts about the world (domain).

⁷ Deductive databases have been defined both with and without integrity constraints in the literature.

In this section, we specifically consider the semantics of the database with respect to the integrity constraint component. We first consider what types of knowledge are expressible via integrity constraints. We then consider various *model based* semantics that have been proposed for, or extended for, ICs. Last, we consider some extensions that have been suggested for ICs that take them beyond first-order.

3.1 Examples of What Integrity Constraints can Express

Often ICs are limited to be formulas in clausal form, as are rules and queries. For ICs in this form, to distinguish them from rules and queries, we write their clauses with ‘ $\Leftarrow$ ’ instead of ‘ $\leftarrow$ ’. Consider the following example with predicate *employee*/5 (a 5-ary relation), which has, say, the attributes *name*, *address*, *salary*, *dname*, and *age*. The domain constraint that states that all employees are over sixteen years old can be written as:

$$A > 16 \Leftarrow \text{employee}(\text{age}: A). \quad 4)$$

We employ for convenience an attribute extended DATALOG, or *fielded* DATALOG, that uses attribute names, as does the relational model, to be understood in the obvious way [49]. This frees us from the inconvenience of notation in Prolog and DATALOG in which attribute names are not expressed, so only the position of a value in an atom indicates which attribute it fills. In fielded DATALOG, the “slots” for any attributes not mentioned in an atom are assumed to be filled by unique (existential) variables, and, of course, the position of a value in the atom is not relevant as the predicate’s slots are named.⁸ Likewise, a convention adopted from Prolog is to write the special variable name ‘ $_$ ’ in any ignored slot—since in Prolog field names do not exist and so the order of the attributes matters—to be interpreted as a unique, existential variable. We shall employ both conventions.

Our next example, seen before in the introduction, expresses the functional dependency of the second attribute on the first in the *employee* relation.

$$\begin{aligned} P_1 = P_2 \Leftarrow & \text{employee}(\text{name}: N, \text{address}: P_1), \\ & \text{employee}(\text{name}: N, \text{address}: P_2). \end{aligned} \quad 1)$$

Consider that we have another predicate *person*/3 which has, say, only the attributes *name*, *address*, and *age*. Thus *employee*/5 is a “sub-type” of *person*/3. The next IC expresses this relationship, that every employee is a person.

$$\begin{aligned} \text{person}(\text{name}: N, \text{address}: P, \text{age}: A) \Leftarrow \\ \text{employee}(\text{name}: N, \text{address}: P, \text{ssn}: _, \text{dname}: _, \text{age}: A). \end{aligned} \quad 5)$$

Alternatively, this statement could instead be made part of the **IDB** as a rule, to generate the *person* relation. Its placement in the **IC** suggests that the facts for the *employee* and *person* relations are obtained separately, and their necessary relationship is expressed by the integrity constraint. Thus, the **IC** is used to *check* the state of the database, not to *generate* any portion of it.

There have been a number of proposals for the syntax of ICs. One is to allow any first-order formula as an **IC**. From the perspective of implementation, however, it may be useful to restrict the syntactic form of ICs. Often ICs are restricted to *denial constraints*; that is, formulas of the form

$$\Leftarrow L_1, \dots, L_n.$$

in which the L_i s are atoms if default negation is not part of the database, and are atoms or (default) negations of atoms if default negation is part of the (normal) database. For the three **IC** given above, the

⁸This field notation for DATALOG is merely syntactic convenience. It is easy to implement, for instance on top of Prolog, and is used by the Carmin system [49].

corresponding denial constraints are:

$$\Leftarrow \text{employee}(\text{age}: A), A \leq 16. \quad 4')$$

$$\Leftarrow \text{employee}(\text{name}: N, \text{address}: P_1), \text{employee}(\text{name}: N, \text{address}: P_2), P_1 \neq P_2. \quad 1')$$

$$\Leftarrow \text{employee}(\text{name}: N, \text{address}: P, \text{ssn}: _, \text{dname}: _, \text{age}: A), \text{not person}(\text{name}: N, \text{address}: P, \text{age}: A). \quad 5')$$

Sometimes extra predicates may need to be introduced to write a constraint as a denial constraint. Consider the following formula that is meant as a referential integrity constraint that expresses that the department name for every employee must appear in the department relation (an inclusion dependency):

$$\forall N, P, S, D, A. \exists R, T. (\text{employee}(N, P, S, D, A) \rightarrow \text{department}(D, R, T))$$

To represent this as a denial constraint, we first add a new predicate *deptname*/1, and the following rule for *deptname*/1:

$$\text{deptname}(D) \leftarrow \text{department}(D, R, T).$$

This rule essentially defines a projection on *department*/3. Then the integrity constraint can be represented as follows:

$$\Leftarrow \text{employee}(\text{name}: _, \text{address}: _, \text{ssn}: _, \text{dname}: D, \text{age}: _), \text{not deptname}(D).$$

Assume *department*/3's attributes are *name*, *region*, and *type*. We could likewise write this using attribute-extended DATALOG, without having to introduce a new predicate:

$$\Leftarrow \text{employee}(\text{dname}: D), \text{not department}(\text{name}: D). \quad 2)$$

Recall we assume of our notation for attribute-extended DATALOG that

$$\text{department}(\text{name}: D) \equiv \exists R, T. \text{department}(\text{name}: D, \text{region}: R, \text{type}: T)$$

Thus, the existential quantification is *inside* the **not** within the integrity constraint, and so the overall quantification is as we intend.

Certain classes of formulas, however, cannot be written in the form of denial constraints. Non-range-restricted ICs, such as

$$\forall X, Y. (q(X, Y) \leftarrow p(X))$$

cannot be written as normal denial constraints.

We also have not considered here the use of *aggregation*—average, maximum, minimum, sum, and such—in integrity constraints, or in rules. Aggregation is also a higher-order concept, and cannot be modeled in first-order logic. Work on ICs involving aggregation can be found in [24].

3.2 Model Semantics

The general notion of an integrity constraint is that it is a logical formula *true* of the database. Usually, it is accepted that the IC formula should be *closed*; that is, it ought to be a *sentence*. As such, it can be answered *yes* or *no*: yes, it is “satisfied” by the database; or, no, it is not. We shall see that this general notion is not so straightforward. We must be more careful by what we mean when we say that the database *satisfies*, or

does not *violate*, its ICs. In fact, there are two prevalent views. One is based on logical satisfiability, and the other on logical entailment. In this section, we look at model semantics and the ways in which one can formalize the meaning of ICs. The situation becomes more complex when we allow default negation in rules, queries, and ICs, since this leads to a non-monotonic logic.⁹ We discuss some of the approaches for defining the meaning of ICs in the context of default negation.

Model semantics offers a way to define formally the *meaning* of a logic database. As discussed in Section 2, there has been much work, and various approaches, to characterize both DATALOG and DATALOG[−] databases via model semantics. Once we introduce integrity constraints into databases, it is necessary to extend any model semantic characterization to cover the meaning of the integrity constraints (with respect to the rest of the database) too. Again, there are a number of approaches.

An *interpretation* over a logical language is a truth assignment (a *valuation*) of *true* or *false* to every constructible ground atom, or (definite) fact.¹⁰ It can be represented as a set: any atom in the set is considered *true*, and any constructible atom not in the set is considered *false*. Every constructible sentence can be then assigned a valuation with respect to the interpretation, preserving truth in the obvious way over the logical connectives and quantifiers.

A *model* M of a collection of sentences, say $\mathcal{P}$ (a DATALOG database is such a collection), is an interpretation I of the language of $\mathcal{P}$ such that the valuation of every sentence in $\mathcal{P}$ is *true*. A Herbrand model of $\mathcal{P}$ is a model of $\mathcal{P}$ that is restricted to only the predicates and constants (and function symbols, in the case of logic programs) that are mentioned in $\mathcal{P}$. (See [7, 67] or another basic text in logic for more background.) We restrict our attention to Herbrand models, and, from this point on, when we say “model”, we mean a Herbrand model.

A model of $\mathcal{P}$ is called a *minimal* model if no subset of it is also a model of $\mathcal{P}$. Consider first a definite DATALOG database $\mathbf{DB}$. As every clause in $\mathbf{DB}$ is a rule or a fact—so has a non-empty head—the *consistency* (that it has a model) of the database is always assured. (Recall that no negated facts can be *deduced* from $\mathbf{DB}$.) Furthermore, it can be shown in this case that $\mathbf{DB}$ has a unique, minimal model. The generally accepted model semantic characterization of $\mathbf{DB}$ is this minimal model. We denote this model by $\mathbf{M}_{\mathbf{DB}}$.

There are two standard approaches to ascribe the meaning of $\mathbf{IC}$. Let $\mathbf{DB} = \mathbf{E}_{\mathbf{DB}} \cup \mathbf{I}_{\mathbf{DB}}$ again for the following discussion. By the *consistency* definition [59], the ICs are *satisfied* iff $\mathbf{DB} \cup \mathbf{IC}$ is *consistent*; that is, it has a model. By the *entailment* definition, the ICs are *satisfied* iff $\mathbf{DB} \models \mathbf{IC}$; that is, the ICs in $\mathbf{IC}$ are consistent with *all* models of $\mathbf{DB}$.

There are problems with both definitions, however. Neither fully captures our intuition of integrity constraints. For the case of DATALOG databases in which inequality is not permitted, when ICs are restricted to be denial constraints, these definitions do not differ [83]. If an IC may be any clause, however, the two definitions may effectively differ.

Under the consistency definition, ICs which are syntactically equivalent to rules (that is, they do not have empty heads) will, indeed, behave as rules. This is because $\mathbf{DB} \cup \mathbf{IC}$ may be consistent, and thus have a model, but this model need not be the intended model, $\mathbf{M}_{\mathbf{DB}}$. (It is $\mathbf{M}_{\mathbf{DB} \cup \mathbf{IC}}$ instead.) Thus, the distinction or value in calling these formulas integrity constraints is lost. So this does not capture our intuition that integrity constraints and rules are epistemically different, and that integrity constraints only “check” the database but do not generate data.

Under the entailment definition, such “rule” ICs intuitively still serve only to deny or confirm the database’s model according to our notion of ICs. However, the entailment definition does not capture our intuition of integrity constraints either, for a different reason. Consider the empty database (with no clauses). All

⁹ We must pragmatically consider the addition of default negation (DATALOG[−]), however, since the relational model allows set minus.

¹⁰ A formula is *constructible* over a language if it can be constructed with the variables, constants, and predicates in the language and the logical connectives and operators.

models are models of the empty database. Therefore, *no* non-tautological IC is logically entailed. This is not what we intend either.

The two definitions differ for denial rules too once normal (DATALOG⁺) rules are permitted, or once inequalities are permitted (for much the same reasons that they differ for “rule” constraints). Consider IC 1 on page 3 which expresses a functional dependency in the relation *employee* that attribute *name* functionally determines *address*. With the use of an inequality, this can be rewritten as a denial constraint as in IC 1' on page 9. Consider the database consisting of the single fact *employee*(*name*: *john*, *address*: *baltimore*).¹¹ A legitimate model, albeit not minimal, of this database is

$$\{\textit{employee}(\textit{name}:\textit{john}, \textit{address}:\textit{baltimore}), \\ \textit{employee}(\textit{name}:\textit{john}, \textit{address}:\textit{washington})\}$$

(Note that we have not made the closed world assumption logically explicit in the database, so this is a model. Also note that, in this example, this is not technically a Herbrand model since ‘*washington*’ was not mentioned in the database. However, one can easily imagine that the database actually contains other unrelated facts which do mention ‘*washington*’, so this then would be a legitimate Herbrand sub-model.) Clearly, this model violates IC 1', so the database is invalid under the entailment definition. However, because the model $\{\textit{employee}(\textit{name}:\textit{john}, \textit{address}:\textit{baltimore})\}$ of this database is consistent with IC 1', the database is valid with respect to the consistency definition.

Likewise, these definitions can differ when normal denial constraints are allowed. This is not surprising, since any “rule” constraint can be rewritten as a normal denial constraint. An example of this is IC 5 on page 8 which can be rewritten as IC 5' on page 9. Consider again the database consisting of the single fact *employee*(*name*: *john*, *address*: *baltimore*). A legitimate model (again, not minimal) of this database is

$$\{\textit{employee}(\textit{name}:\textit{john}, \textit{address}:\textit{baltimore}), \textit{person}(\textit{name}:\textit{john})\}$$

In this case, the minimal model of the database is inconsistent with respect to IC 5', thus, the database is invalid with respect to the entailment definition. However the model above satisfies the IC, so the database is valid with respect to the consistency definition. (This example is simplistic since one would need to consider an appropriate semantics for DATALOG⁺ if one is to permit normal denial constraints or normal rules.)

A standard way to reconcile the two approaches is to say that the ICs must be satisfied by the database's minimal model, M_{DB} :

$$M_{DB} \models IC$$

This works fine for definite (thus non-disjunctive) databases without default negation (DATALOG) and without inequalities. The picture becomes complicated, however, when we consider databases which contain disjunction, which contain default negation (DATALOG⁺), or which contain inequalities, as demonstrated above. Furthermore, in the case of normal or disjunctive databases, there may be no unique minimal model to represent the database's meaning. Instead, there may be a number of *minimal* models of the database. One approach is to consider the *set* of a database's minimal models to characterize the database. Again, the entailment and consistency approaches differ. The pure entailment approach is that *every* minimal model of the database must satisfy IC. The pure consistency approach is that at least *one* minimal model satisfies IC. In the latter case, the ICs effectively eliminate those models which do not entail them. There exist possible semantics for ICs between the entailment and consistency definitions that are sensible ([13] and [84]), but we do not consider them here.

Thus once we allow disjunction or default negation in our databases, we may lose the substance of our intuition for integrity constraints, that they only “check” the database and do not contribute to generate data. Any proof procedure, or query evaluation strategy, must employ the ICs, as the ICs effectively eliminate models from consideration. Our intuition is still weakly satisfied: the ICs do not actively generate data, but

¹¹ We take liberties by not filling in all the attributes.

may indirectly allow for more to be inferred.

Integrity constraints and negation are closely related. Denial constraints are effectively a way to add logical negation into deductive databases. Recall that the headless clause is equivalent to a disjunct of negated atoms. As seen with the consistency and entailment definitions, a database with denial constraints need not be consistent; that is, there may be no model that satisfies both the database's state and its constraints. In [32], it is shown that any normal logic program (or deductive database) can be transformed into a disjunctive logic program (or disjunctive deductive database) with certain associated integrity constraints added. The subset of the set of minimal models of the disjunctive logic program that are also consistent with the ICs is equivalent to the set of stable models [42] of the initial normal logic program. This is a powerful result, and yields credence to the fact that integrity constraints and negation are two different approaches to many of the same semantic issues.

3.3 Extensions to the Basic Model

The ICs considered so far are *universal constraints*: updates preserve their truths. It is also possible to define the concept of a *state constraint* to be an IC formula *true* over the current state of the database, but whose truth is not necessarily preserved by updates. State constraints can be useful for the applications of ICs discussed in Section 4.

Another extension is a constraint that a user may request for query processing. For example, in a flight database a user may wish to request only flights for which no change of planes is required. Clearly, many flights may still involve a change of planes, but they will be eliminated from consideration. Related to such a *user constraint* is the concept of a *preference constraint* by which a user expresses a preference concerning possible answers. Such preferences may be expressed by annotated user constraints [36].

All the ICs considered so far have been *static constraints*: they refer to a given state of the DB. *Temporal constraints* represent constraints across **DB** states. Various approaches to temporal ICs are discussed in [19].

The use of modal logic has also been considered for expressing queries and ICs [83]. In this approach, ICs represent knowledge about what the database knows, rather than knowledge about the actual world. In [26], both ICs that express knowledge about the world and about the database itself are allowed. The use of modal logic for expressing ICs is discussed in [89].

4 Reasoning with Integrity Constraints

We advocate the explicit representation of the database's integrity constraints (in **IC**), just as the database's rules and facts. Then it is explicitly clear what the constraints of database are. When the integrity constraints are left implicit, it is not necessarily obvious what the semantics of the database is, and the semantic information made available by the ICs is not available to applications.

There is, however, the separate issue of how one should best reason over a database with integrity constraints. It might be easier, in certain contexts or for certain applications, to reason about a database in which integrity constraints have been processed partially, or eliminated altogether. A transformation of a database in such a way should result in a semantically equivalent database, of course. That is, every query posable to the original database should result in the same set of answers when posed to the transformed database. Such reasoning techniques are important not only for applications which employ integrity constraints, but also may be necessary as part of the query evaluation itself.

We present three techniques for processing integrity constraints. The most general one, presented in [60], describes the transformation of a deductive database (or logic program) with integrity constraints into an

equivalent one without constraints. The result of this transformation is a normal deductive database (or logic program) with some of the original deductive rules modified via “incorporating” (or “compiling-in”) the information of the integrity constraints into them. In [30], all integrity constraints are expressed as denial constraints (as described in Section 3.1). The denial constraints are used to eliminate semantically meaningless minimal models—those models which are inconsistent with the denial constraints—in disjunctive deductive databases. The method described in [12] compiles out the integrity constraints too. The approach does not eliminate integrity constraints by rewriting deductive rules however, but rather it annotates the rules with relevant pieces of the integrity constraints called *residues*, to be used during query evaluation.

4.1 Eliminating Integrity Constraints

The transformation of Kowalski and Sadri [60] not only eliminates integrity constraints, but also makes the modified database consistent with them. When the original theory is inconsistent, the new theory represents a particular way of restoring consistency. The transformation, which is applicable to any deductive theory with definite rules, is done as follows:

Let $\leftarrow a(\vec{x}), L_1, \dots, L_n$ be an integrity constraint (expressed as a denial constraint) to be eliminated, in which $a(\vec{x})$ is an atom and $L_1, \dots, L_n$ are atoms or negated atoms. (Thus, the rules are in DATALOG⁺.) Then each deductive rule of the form

$$a(\vec{y}) \leftarrow K_1, \dots, K_m. \quad (6)$$

in which $K_1, \dots, K_m$ are literals, and for which $a(\vec{x})$ and $a(\vec{y})$ unify with most general unifier θ [67], is replaced by at most three new rules. Intuitively, the first new rule deals with the case when the integrity constraint is applicable to rule (6). The next two rules, potentially to be added, handle the case when it is not applicable because X and Y do not unify. The first rule is

$$a(\vec{y})\theta \leftarrow K_1\theta, \dots, K_m\theta, \mathbf{not} (L_1, \dots, L_n)\theta.$$

Again, we take liberties with our use of default negation here to allow it to scope over a conjunction of atoms. The next two rules are needed only when $a(\vec{y})\theta$ is a non-trivial instance of $a(\vec{y})$ (that is, not a variant of $a(\vec{y})$). The first of these is

$$a(\vec{y}) \leftarrow K_1, \dots, K_m, \mathbf{not} t(\vec{y}).$$

in which t is a new predicate symbol, and there is a single rule defined for it as follows.

$$t(\vec{y}) \leftarrow (\vec{y} = \vec{x}).$$

Here, $(\vec{y} = \vec{x})$ represents a conjunction of equalities. Given $\vec{y} = Y_1, \dots, Y_n$ and $\vec{x} = X_1, \dots, X_n$, then $(Y_1 = X_1), \dots, (Y_n = X_n)$ is meant. If $n = 0$, the negation of the empty conjunct is definitionally *false*, in which case rule 6) does not need to be added. If $n > 1$ (so the conjunct contains more than one literal), the rule may be written instead as a collection of deductive rules, employing a technique similar to the one used in Section 3.1 to rewrite an arbitrary integrity constraint as a denial constraint.

It is easy to show that the transformed database, call it **DB'**, is always consistent with the original set of integrity constraints **IC**, in the sense that there exists no SLDNF refutation [67] of any variable-free instances of clauses in **DB'** $\cup$ **IC**. It can also be shown that the original database **DB** with **IC** is equivalent to **DB'**, in the sense that any arbitrary query results in same set of answers for either database (the original or the transformed).

There are several problems inherent in this approach, however. First, by incorporating ICs into **IDB** rules, the epistemic distinction that is usually drawn between ICs and rules (as described in Section 3) is lost. Consider (6) and assume that a was an extensional predicate (that is, $m = 0$). Eliminating the integrity

constraints as described above results in adding a rule for a , thus making it an intensional predicate. Second, if an IC that is to be eliminated contains extensional predicates only, then some of the $\mathbf{E}_{\mathbf{DB}}$ facts would have to be expressed as rules in the transformation. Since a database user may still want to update such facts, the update would now have to be a view update. Third, incorporating ICs into rules introduces default negation into the transformed database; this may not be desired when the original database does not use default negation.

The following two subsections describe two techniques for eliminating integrity constraints that largely avoid the above problems. The first technique allows for the complete elimination of ICs in the restricted domain of disjunctive databases, in which disjunctive facts are represented via minimal models. In the second approach, ICs are eliminated not by explicit rule rewrite, but by introducing a new type of information stored *with* rules, called *residues*.

4.2 Model Elimination

In normal and disjunctive databases, knowledge is often considered synonymous with the set of *canonical* models that characterize the semantics of the database. For a normal database, the canonical models might be its stable models (if the stable model semantics has been chosen), or, for a disjunctive database, its minimal models. The semantics for a definite database is often determined by a unique minimal model, and so the consistency and entailment definitions of ICs (as presented in Section 3.2) coincide, and the ICs can be disregarded. Given a normal or disjunctive database and a set of integrity constraints that is consistent with the database, only a subset of the database's canonical models should be considered (under the consistency approach). If the canonical models of the database are not explicitly known—and most proof procedures and query evaluation strategies do not manifest the models—it is necessary to query the database unioned with the ICs.

Thus under the consistency interpretation, integrity constraints must play an active role in the deductive process. Models that do not satisfy an integrity constraint are *eliminated* as they are not considered to be a part of the semantic characterization of the database. The possible reduction of the number of models describing the semantics of a database implies the possibility that more information can be deduced from it. ICs play the role of filters to eliminate models that are not meaningful in the database. Consider the following example [30].

Example 1. Let the clause $\Leftarrow a(X), b(X)$ be an integrity constraint and let $\mathbf{DB} = \{a(2) \vee b(2), a(1), b(1) \vee b(2)\}$ for which the semantics is represented by the set of its minimal models:

$$\left\{ \{a(1), a(2), b(1)\}, \{a(1), b(2)\} \right\}$$

The model $\{a(1), b(2)\}$ is the only one that satisfies the constraint; hence, $\mathbf{DB}$ does not entail the constraint, but is consistent with it.

Note that if the query $\Leftarrow b(X)$ is asked, we should get the answer $b(2)$. This requires use of the IC. Without the IC, we would get no answers, as the intersection of the two minimal models is empty with respect to predicate b .¹²

As long as the **IC** contains indefinite predicates only (that is, predicates used to express disjunctive facts), they can be eliminated by removing models in which they are not satisfied.

In logic programming for disjunctive theories, a new inference rule must be added for the query processing proof procedure to be *complete*. For example, the SLI proof procedure extends the traditional SLD procedure with factoring and reduction to accomplish this. (See [68].) An extra inference rule named *model*

¹²In a disjunctive logic programming system (or a disjunctive deductive database system) which provides *disjunctive answers* (see [68]), the weaker disjunctive answer $(b(1) \vee b(2))$ would be returned.

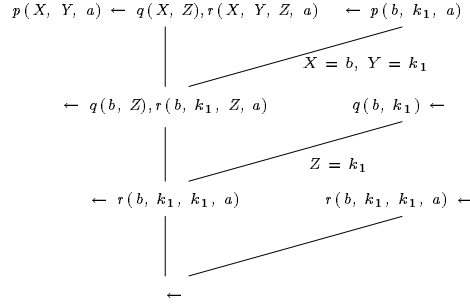


Figure 1: Refutation tree for subsumption.

elimination in [69] also allows for complete inferencing over disjunctive theories, and will also account for denial constraints, and is implemented and discussed further in [86]. This work also applies to disjunctive databases with integrity constraints.

4.3 Residue Method

The basic idea behind the residue method [10, 12] is the use of a theorem proving technique called partial subsumption to attach integrity constraint fragments called residues to relations and rules. Hence, we start with a presentation of the concept of subsumption. A clause C_1 *subsumes* a clause C_2 if there is a substitution θ such that $C_1\theta$ is a sub-clause of C_2 . For example, if

$$C_1: p(X, Y, a) \leftarrow q(X, Z), r(X, Y, Z, a).$$

and

$$C_2: p(b, Y, a) \leftarrow q(b, Y), r(b, Y, Y, a), s(a).$$

then C_1 subsumes C_2 by the substitution $\{X/b, Z/Y\}$.

To illustrate partial subsumption, we show the basic procedure to test for subsumption between C_1 and C_2 . First, the clause we are testing to be subsumed— C_2 in this case—is instantiated to a ground clause via a substitution θ which uses all new constants $k_1, \dots, k_n$, not present in C_1 or C_2 . This results in

$$C_2\theta: p(b, k_1, a) \leftarrow q(b, k_1), r(b, k_1, k_1, a), s(a).$$

in which $\theta = \{Y/k_1\}$. Next, the clausal formula $C_2\theta$ is negated, resulting in the conjunction of one denial rule and three facts:

$$\neg C_2\theta : \{(\leftarrow p(b, k_1, a)), q(b, k_1), r(b, k_1, k_1, a), s(a)\}.$$

The final step is the construction, if possible, of a linear refutation tree, with C_1 as the proposed subsuming clause, the root, using at each resolution step only clauses from $\neg C_2\theta$. It is known that C_1 subsumes C_2 *iff* there is a refutation tree that ends with the null clause. In our example, such a tree exists, and is shown in Figure 1.

The subsumption algorithm can be applied to determine *partial subsumption*, with an integrity constraint taking the part of the subsuming clause and a relation (negated) taking the part of the subsumed clause. In general, the null clause is not obtained, because integrity constraints rarely subsume relations.¹³ However, an IC might partially subsume a relation, leaving a fragment at the bottom of the refutation tree. Such a

¹³There is something odd about the database's design if they do.

fragment is called a *residue*. A residue, if nontrivial, represents an interaction between a relation and an integrity constraint. In general it is computationally complex to perform partial subsumption. However, in practice, integrity rules do not contain more than one instance of any predicate. In this event, partial subsumption can be performed in linear time according to the number of predicates in the integrity constraint.

Before commencing subsumption, constants which occur in ICs must be handled specially as the following illustrates. Let $r(X, Y, Z)$ be a relation and $X = a \Leftarrow r(X, b, Z)$ be an integrity constraint. The subsumption algorithm cannot work in this case, because the IC's clause cannot be resolved with $r(k_1, k_2, k_3)$ (the instantiation of $r(X, Y, Z)$). A transformation called *expansion* is applied to the IC which allows for the subsumption to proceed. In this case, expanding $(X = a) \Leftarrow r(X, b, Z)$ yields $(X = a) \Leftarrow r(X, Y, Z), (Y = b)$, which *can* be resolved with $r(k_1, k_2, k_3)$ to yield $(k_1 = a) \Leftarrow (k_2 = b)$. This is not yet the residue. The grounding substitution θ must first be reversed by applying θ^{-1} . The residue is then $(X = a) \Leftarrow (Y = b)$, which intuitively represents the integrity constraint's effect on the predicate r .

In some cases an extra step, called a *reduction*, is necessary for obtaining a useful residue. This step basically takes care of removing redundant atoms. We leave out the details of expansion and reduction here.

Next, we state the definitions of partial subsumption and residue.

Definition 4.1. We say that IC *partially subsumes* the atom A , iff IC does not subsume $\Leftarrow A$, but a sub-clause of IC^+ (in which IC^+ is a result of expanding IC) subsumes $\Leftarrow A$.

Definition 4.2. Given an integrity constraint IC and atom R , apply the subsumption algorithm to IC^+ and $\Leftarrow R$ until no more resolutions are possible. Let C be the clause at the bottom of a refutation tree. Then $(C^-)\theta^{-1}$ (in which C^- is a result of reducing C) is a *residue* of IC and R .

Several different residues can result between an integrity constraint and a relation. If the residue is IC^- , that is, there was no resolution, we call it a *maximal* residue. We call a residue that is always *true*, such as $(a = a)$, a *redundant* residue. Finally, we call an integrity constraint *merge-compatible* with a relation if there is at least one residue that is neither maximal nor redundant. These are the residues that can be used in semantic query optimization, as described in Section 5.1 (Example 2). Multiple residues may arise from the same constraint or from different constraints. When this happens, heuristics may be used to select the residue to be used [38].

5 Applications of Integrity Constraints

As integrity constraints are used to represent the semantics of the database, they are useful in many database applications, such as the design of relational databases [88]. Certainly, ICs are vital for maintaining the consistency of the database, and for updating the entries in the extensional database. This is the original reason why integrity constraints were introduced. In this section, we describe other important applications, which include semantic query optimization, cooperative query answering, view updates (updating intensional relations), the use of ICs to combine databases meaningfully and to resolve inconsistencies, and several additional applications.

5.1 Semantic Query Optimization

Semantic query optimization (SQO) is a technique which uses integrity constraints to improve the efficiency of query evaluation. Most of the SQO algorithms are applied to a query after the query is posed to a database, but before the syntactic optimization is performed. SQO typically consists of two steps. During the first step several queries logically equivalent (with respect to $\mathbf{I}_{DB}$ and $\mathbf{IC}$) to the original query are generated. Then, the query with the lowest estimated evaluation cost is chosen and submitted for evaluation. This

step is based on heuristics [12] which take into account physical database design (indices, table sizes, and so forth). Ideally, all these aspects should be integrated into one, coherent optimizer.

The major types of SQO are:

1. *Join elimination*: this transformation eliminates a join from the query.
2. *Restriction introduction/scan reduction*: the introduction of restriction on, for instance, an indexed attribute may reduce the cost of evaluation.
3. *Restriction elimination*: this can be done when restriction on one of the attributes implies restriction on another attribute.

Work on SQO has been done by several researchers starting with [70], [55], and [58]. The most influential has been the SQO technique developed in [11], [10], and [12]. The main idea of this technique can be summarized as follows. SQO is done in two phases, the *compilation* phase and the *transformation* phase. During the compilation phase residues (see Section 4.3) are computed and associated with relational tables and rules (views). The result of the compilation phase is a set of semantically constrained rules, containing all the information carried previously by integrity constraints. During the second phase, which takes place after a query is given to the system, the semantic transformer uses these stored residues to generate semantically equivalent queries that may be processed more efficiently than the original query.

The following example describes the method.

Example 2. Let the database contain the following relations: *student* with attributes *id* and *name*, *takes_section* with attributes *student_id* and *name*, *faculty* with attributes *id*, *section*, and *age*. Assume also that there is an integrity constraint which says that all faculty members are more than thirty years old. That is,

$$(A \geq 30) \Leftarrow \text{faculty}(\text{id}: _, \text{section}: _, \text{Age}: A). \quad 7)$$

By the method of partial subsumption described in Section 4.3, with the above IC and the relation *faculty*, we can construct the residue: $\{A \geq 30 \Leftarrow \}$, which then annotates the relation *faculty*. Intuitively, this means that any query with the predicate *faculty* must satisfy the condition stated by the residue to succeed.

The query below asks for the names of all students taught by professors who are older than twenty.

$$\begin{aligned} \text{query}(N) \Leftarrow & \text{student}(\text{id}: I, \text{name}: N), \\ & \text{takes_section}(\text{student_id}: I, \text{section}: S), \\ & \text{faculty}(\text{id}: _, \text{section}: S, \text{age}: A), A > 20. \end{aligned}$$

Since the query contains the predicate *faculty*, the residue applies, and the following semantically equivalent query is produced:

$$\begin{aligned} \text{query}'(N) \Leftarrow & \text{student}(\text{id}: I, \text{name}: N), \\ & \text{takes_section}(\text{student_id}: I, \text{section}: S), \\ & \text{faculty}(\text{id}: _, \text{section}: S, \text{age}: A). \end{aligned}$$

The *select* operation on the **Faculty** table is not necessary, since all tuples satisfy it. Thus, the query can be potentially evaluated in a more efficient way.

The primary advantage of the residue approach is that by associating integrity constraints explicitly with the rules reduces the search space of integrity constraints that can apply to a given query. This allows for most of the computation related to semantic query optimization (SQO) to be done at compile time. However, this rigid combining of rules and integrity constraints has disadvantages. It works well for queries that can be represented succinctly as a union of conjunctive queries, in which case it is sufficient to consider residues

attached to each rule in isolation. In the case of recursive rules, however, residues have to be considered with respect to the derivation trees produced during evaluation. What is required in the recursive case is the rewriting of the evaluation, not simply individual rules. Research in SQO has been advanced in this direction by [12], [61], [63], and [66].

Another case for which the residue technique must be adapted—or another approach found—is for query evaluation that interleaves join and union operations. (Bottom-up query evaluation does this.) Once again, such a query cannot be expressed succinctly as a single union of joins, to which the residues could be directly attached. Research in SQO for queries involving an arbitrary nesting of union and join operations has been initiated in [64] and developed further in [45], [47], and [48].

SQO can be extended to object-oriented databases. An approach recently developed [50] transfers existing SQO techniques (for instance, based on residues) from the logic (or relational) to the object database model. The core of this technique is the transformation of an object schema into a relational schema, in such a way that the object semantics is reflected via integrity constraints at the relational level. (See Section 5.5.) Correspondingly, each object query is transformed to a query over the relational database. The optimization of a query is performed at the relational level (as described above) to produce a new relational query, which is then mapped to a new, optimized object query. This technique works for a large class of object queries, in particular, queries with methods and structure constructors.

5.2 Cooperative Answering

Cooperative answering augments database systems to respond with more than just the query's answer set, whenever such additional information would help to disambiguate, clarify, or contribute to the user's goals and intentions. Numerous cooperative answering techniques have been explored. (See [35] for a survey.) The techniques vary by which aspect of the query/answer cycle they address—for instance, to attempt to clarify a user's *misconceptions* as indicated by the query, or to help the user formulate a query—and by what type of additional response they provide—for instance, an explanation of the misconception, or a list of candidate queries. Cooperative answering provides relevant *knowledge* in addition to the *data* of the answer set. This knowledge is derived from the semantics of the database. The database's semantics, in turn, is represented via its integrity constraints.

Work in cooperative answering has built upon previous work on semantic query optimization. In [38], [39], and [40], it was shown that sometimes a query can be known to *fail* (to produce the empty answer set), without evaluating the query. This is true whenever the query is subsumed by an IC; that is, any answer to the query would be inconsistent with respect to the database.

When one asks a query, one usually wants answers. Thus, when a query *fails*, the empty response is unexpected. Furthermore, the empty response is ambiguous as it does not indicate *why* the query fails, or what about the query is responsible for the failure. (For instance, a part of the query may result in failure.) When a query *must* fail, because it is inconsistent with respect to the semantics of the database, it indicates a *misconception* on the user's behalf. In such cases, we often call the query itself a misconception. The query is guaranteed never to succeed on that database, unless the semantics of the database changes. Therefore, it is cooperative to explain the cause of the misconception to the user. Consider the following query:

$Q: \leftarrow in_patient(name: P, ward: maternity),$
 $patient(name: P, gender: male),$
 $infected(patient: P, disease: D),$
 $disease(name: D, contagious: high, type: streptococcus).$

The query asks for any male patient on the maternity ward who has a highly contagious streptococcus infection. Let the database have the following natural integrity constraints:

$$\begin{aligned}
& patient(name: P, gender: female) \Leftarrow in_patient(name: P, ward: maternity). \\
& X = Y \Leftarrow patient(name: P, gender: X), \\
& \quad patient(name: P, gender: Y).
\end{aligned}$$

The second IC expresses the functional dependency in *patient*. The first IC may have been added for the purpose of double-checking patient record entries in the maternity ward. With this knowledge, it is possible to prove logically that the query Q must fail.

Gal and Minker studied misconceptions. They could determine whether a query was a misconception by checking whether it was subsumed by an IC, and so was inconsistent with the database. They built a mechanism for the misconception test using the technique of residues and partial subsumption (presented in Section 4). The subsuming IC provides a reason why the query fails. The proof of subsumption and the IC are then used as the basis of an explanation to the user. See [33] and [43] for further developed approaches to misconception detection and explanation.

User constraints mentioned in 3.3 were introduced as a cooperative technique to help focus the response to a user's queries to the user's interests [33, 37]. While they are not ICs—they are not necessarily consistent with the database as ICs are required to be—they can be employed as such by the algorithms that test for misconceptions to remove portions of the answer set in which the user is not interested (as indicated by the user constraints). This is a variation on ICs that can allow for user-specific semantics to be added to the database.

A second type of cooperative response is an *intensional answer* (IA), another query formula. Intensional answers have been alternatively characterized as formulas which logically subsume the query (so the intensional answer's answer set is a superset of the query's), which are logically equivalent to the query, or which are logically subsumed by the query. The latter two characterizations are more prevalent. An intensional answer is warranted when the intensional answer is simpler than the query itself, or explicates something about the query and database.

Intensional answers were introduced by [56] in which IAs are viewed as rewrites of a query which preserves its semantics (and, thus, is equivalent to the query). In [16] and [17], IAs are considered under the sufficiency criterion (that is, the IA is subsumed by the query). While intensional answers can be considered without integrity constraints, ICs allow more IAs to exist: the query and IA are logically equivalent with respect to the ICs and the database. In [75] and in [78] and [79], intensional answers are considered based on residues. It may be argued that IAs which are equivalent to the query with respect to integrity constraints are more meaningful because their equivalence conveys semantic information. Consider the following query:

$$\begin{aligned}
& \leftarrow course(title: _, semester: "Spring 1997", professor: P), \\
& \quad employee(name: P, rank: professor, dname: "Computer Science").
\end{aligned}$$

This asks for all the professors in computer science who are teaching a course this semester. Consider that the following integrity constraint exists:

$$\begin{aligned}
& course(semester: S, professor: P) \Leftarrow \\
& \quad employee(name: P, rank: professor, started: B), \\
& \quad semester(S), B \leq S.
\end{aligned}$$

This states that every professor must teach at least one course a semester. If so, the query above is equivalent to the set of *all* computer science professors,

$$\leftarrow employee(name: P, rank: professor, dname: "Computer Science").$$

Thus, it could be returned as an IA. This indicates that the user has a misconception. Not to respond with the IA above—that all professors teach every semester—would be misleading. The user probably will miss the fact that the answer set is all the computer science professors. Even if the user noticed this, somehow,

the user could not know that it is significant. The knowledge was left out.

Other techniques for cooperative answering “create” knowledge in an attempt to disambiguate or explain the situation for the user. One such technique is to identify the *false presuppositions* of a failing query to the user. In the case of conjunctive queries,¹⁴ this is synonymous with finding the *minimal failing sub-queries* (MFSs) of the query [44]. (We advocate doing this when the query failed *and* was first determined *not* to be a misconception. Otherwise, the misconception ought to be explained instead [49].) It is cooperative to focus on the part of the query that fails. Thus the user knows what information is not in the database and can appropriately revise what he or she is seeking. Consider a valid variant of the misconception query from before:

$$Q: \leftarrow \text{in_patient}(\text{name}: P, \text{ward}: \text{maternity}), \\ \text{infected}(\text{patient}: P, \text{disease}: D), \\ \text{disease}(\text{name}: D, \text{contagious}: \text{high}, \text{type}: \text{streptococcus}).$$

Assume that this query fails (but is not a misconception). It may be that the sub-query

$$Q: \leftarrow \text{in_patient}(\text{name}: P, \text{maternity}), \\ \text{infected}(\text{patient}: P, \text{disease}: D), \\ \text{disease}(\text{name}: D, \text{contagious}: \text{high}).$$

fails. That is, there are currently no patients (on any ward) with a highly contagious disease.

The cooperative technique of false presuppositions does not employ integrity constraints; it is a technique used when there is no semantic knowledge to bring to bear on the failing query. This side of cooperative answering is closely related with *knowledge discovery in databases*. In [44], the complexity of finding MFSs of a query is determined, and an optimal algorithmic approach for discovering MFSs is devised. Note that the “semantics” that are discovered by the process take the same form as integrity constraints.

Thus while the MFSs found for a query are not ICs, they could be. Indeed, a database designer may discover that he or she had intended some MFS as an IC in the database, but the IC had been missed in the specification. In this case, a discovered MFS could be “promoted” to be an IC. In any case, an MFS may be treated as a *state constraint* (Section 3.3), at least until a transaction invalidates it. State constraints can be useful for efficiency, as they can be used by any semantic query optimization routines just as any IC. (Recall the previous section.)

In this section, we stated that a query is considered a misconception whenever it is “inconsistent” with the database. The intent is that the query cannot have answers with respect to the database, and so the query formula could serve as an integrity constraint. Thus the semantics of misconception is linked with the semantics we chose for integrity constraints. Since there are choices for such semantics, there are choices for the semantics of cooperative answering too.

5.3 Combining Databases and Resolving Inconsistencies

During the past several years, there has been considerable work devoted towards answering queries across heterogeneous and distributed databases. This work is critical for organizations located remotely from one another, but where related data are stored, and for organizations whose data are distributed simply for organizational reasons. Given a set of deductive databases and a set of integrity constraints that are considered to be the **IC** for every database, we wish to “combine” these databases to obtain answers to queries. If one ignores the integrity constraints, then the combination of the databases via their union is easily shown to be consistent. However, when one accounts for the **IC**, the union of the individual databases (the merged database) together with the **IC** may be inconsistent.

¹⁴ These are synonymous with basic multi-join SQL queries.

We consider a merging technique that ensures consistency with the **IC**, guarantees in a certain sense minimal loss of information to accomplish this, and results in a potentially disjunctive database. The goal is to combine a set of DATALOG databases, $\{\mathbf{DB}_1, \dots, \mathbf{DB}_n\}$ with **IC** to obtain a maximal set of consistent data. We allow the resulting database to be disjunctive. For example, consider two DATALOG databases, $\mathbf{DB}_1 = \{A\}$ and $\mathbf{DB}_2 = \{B\}$, and $\mathbf{IC} = \{\Leftarrow A, B\}$. There are three consistent and correct combinations of $\mathbf{DB}_1$ and $\mathbf{DB}_2$: $\{A\}$, $\{B\}$, and $\{(A \vee B)\}$. The last is maximally consistent, and we choose it as characterizing the combination. Thus, the combination consists of two minimal models—*models* $\{A\}$ and $\{B\}$ —and any answer to a query must be correct with respect to both of these minimal models.

We define the *consistency* of a set of DATALOG databases as follows. A database **DB** (which may be disjunctive) is said to be *consistent* with respect to a set of integrity constraints **IC** *iff* every minimal model of **DB** satisfies **IC**. An algorithm to combine DATALOG databases is given in [2]. We provide an example that illustrates the problem, and we present briefly the steps (as performed by the algorithm in [2]) needed to combine the databases in the example.

Example 3. Let $\mathbf{DB}_1 = \{p(a), p(b)\}$ and $\mathbf{DB}_2 = \{p(c)\}$. (So both databases just contain facts and no rules, in this example.) Consider the following IC:

$$\Leftarrow p(X), p(Y), p(Z), X \neq Y, Y \neq Z, X \neq Z. \quad 8)$$

Databases $\mathbf{DB}_1$ and $\mathbf{DB}_2$ are each individually consistent with IC 8. However, the simple union of $\mathbf{DB}_1$ and $\mathbf{DB}_2$ is not.

Step 1: The union of $\mathbf{DB}_1$ and $\mathbf{DB}_2$, call it **DB**, is $\{p(a), p(b), p(c)\}$. Database **DB** unioned with IC 8 is not consistent. If one were to ask IC 8 as a query, one answer would be the substitution $\{X/a, Y/b, Z/c\}$. Call this substitution θ .

Step 2: The atoms in IC 8 substituted via θ are moved to the left of the clausal arrow, resulting in

$$\neg p(a) \vee \neg p(b) \vee \neg p(c) \leftarrow .$$

(Note that when the atoms in IC 8 substituted via θ are moved to the left, $a = b$, $b = c$, and $a = c$ can be safely eliminated from the above disjunct because they evaluate to *false*.) The negated atoms are renamed to be positive atoms by introducing a new predicate for each involved. For instance, in this case we introduce p' for p with the intended meaning that $p'(a) \equiv \neg p(a)$, for any a . Thus, the formula is rewritten to

$$p'(a) \vee p'(b) \vee p'(c).$$

Next, the minimal models of the disjunctive facts (just a single fact here) are found. They are:

$$\{\{p'(a)\}, \{p'(b)\}, \{p'(c)\}\}$$

Step 3: These minimal models in terms of the original predicates denote the models $\{p'(a), p(b), p(c)\}$, $\{p(a), p'(b), p(c)\}$, and $\{p(a), p(b), p'(c)\}$. Projecting out the “negated” atoms, the models are $\{p(b), p(c)\}$, $\{p(a), p(c)\}$, and $\{p(a), p(b)\}$. These models are the *maximal models* for database **DB** that are consistent with IC 8.

Step 4: The database $\mathbf{DB}'$ is the collection of disjunctions formed by choosing an atom from each of the maximal models. Database $\mathbf{DB}'$ is called the *combined database*. Thus, in this example,

$$\mathbf{DB}': \{p(a) \vee p(b), p(a) \vee p(c), p(b) \vee p(c)\}$$

Note that $\mathbf{DB}'$ is consistent with IC 8.

The algorithm in [2] also works for databases with rules. With some modifications, as explained in [2],

one can always develop a maximal combined database that is consistent with a set of ICs for any definite (DATALOG) stratified database, or for any definite normal (DATALOG⁺) stratified database. In [3], it is shown how to combine databases that consist of first order theories, while in [4], it is shown how to combine default logic databases. The problem of combining DATALOG databases that have both integrity constraints and priorities is solved in [80].

It is a practical necessity to resolve semantic conflicts in the merger of multiple databases with respect to the global integrity constraints simply because such semantic conflicts are assured to arise. Different organizations may own and update the different databases being merged, and so there is no mechanism that can assure that the collection is always consistent. This idea can be taken a step further. Why not allow that a given database itself might be inconsistent with respect to its (or global) integrity constraints? As databases become more complex and very large, it may become inevitable that we accept that a database will have inconsistencies. (Some ICs will still be used to check updates, but others may not be because, perhaps, it is too expensive.)

In [26], mentioned in Section 3.3, they consider that the database may be inconsistent with respect to the integrity constraints, and can tell if an answer from the database should be considered *valid*. In [8], a way to use the ICs to mask out local inconsistencies in a logic program or deductive database is considered. All the techniques discussed above for merging databases in a globally consistent manner can be applied to the problem of presenting a consistent view of a single inconsistent database.

5.4 View Updates

In deductive or relational databases, views are *intensional predicates*, which appear only in the heads of **IDB** rules but do not appear in the **EDB**. An intensional predicate is defined in terms of other intensional predicates and extensional predicates. We consider the problem of view updates—updates to an intensional predicate—in deductive databases. An update to an intensional predicate can only be accomplished by modifying appropriately the underlying extensional predicates, in the *extensional* part of the database. The problem is that there may be more than one way the underlying tables might be modified to effect the update. Which to choose is unclear. This problem of ambiguity is known as the *view update problem*.

Even in the case of a purely definite deductive database, it is unclear how the extensional relations should be modified to accomplish a given view update. For example, consider the database **DB**

$$\begin{aligned} p(X) &\leftarrow a(X) \\ p(X) &\leftarrow b(X) \end{aligned}$$

(in which p is an intensional predicate, and a and b are extensional predicates). Suppose we wish to update the database with the fact $p(c)$. If we are restricted to definite clauses, there are three plausible ways to accomplish this:

- add $a(c)$;
- add $b(c)$; or
- add both $a(c)$ and $b(c)$.

Either of the first two options seem arbitrary. (Why choose one arbitrarily over the other?) The third option, however, results in an update that is too strong. (There is no reason to believe *both* $a(c)$ and $b(c)$.) If we allow disjunctive information in the database, however, we can accomplish the update via a fourth option, by adding $(a(c) \vee b(c))$. This last option appears intuitively to be correct.

In [51], general algorithms for accomplishing view updates (both for insertions and deletions) in definite, and disjunctive deductive databases are provided. Semantic justification of the updates that these algorithms accomplish is given, which makes precise the sense in which they represent “minimal modifications” to the

underlying database. Two kinds of updates are considered: those that involve the insertion or deletion of information into and out of a definite, or disjunctive, deductive database; and those that involve the insertion or deletion of information into and out of a stratified normal disjunctive database. Insertions into normal deductive databases may require the insertion of negative information. (Of course, a database cannot contain negative information explicitly. However, it can yield negative information through inference—non-monotonic reasoning techniques—such as closed world reasoning ([82] and [72]).

An extension to the work in [51] is made in [31], in which a model theoretic approach is introduced which encompasses a wide class of Herbrand semantics, including the perfect model and stable model semantics, for disjunctive databases including negation. The databases may contain disjunctive rules and denial constraints, and may be required to satisfy integrity constraints. They prove the algorithms provided to be correct, and “best”, with respect to the criteria of causing minimal change to the database and giving priority to minimizing deletions.

We illustrate the application of ICs in the framework of [31]. Consider the following propositional **DB**:

$$\begin{array}{ll}
 w \leftarrow \mathbf{not} \ s. & \\
 s \vee t \leftarrow \mathbf{not} \ p, \mathbf{not} \ r. & \\
 p \vee q \leftarrow a. & a \vee b. \\
 q \vee r \leftarrow b. & a \vee c. \\
 p \vee r \leftarrow c. &
 \end{array}$$

Note that the extensional atoms are a, b, c , and the intensional atoms p, q, r, s, t , and w . This database has two minimal models projected over its $\mathbf{E}_{\mathbf{DB}}$: $\{a\}$ and $\{b, c\}$. With the $\mathbf{I}_{\mathbf{DB}}$ considered, the database has five minimal models: $\{a, p, w\}$, $\{a, q, s\}$, $\{a, q, t, w\}$, $\{b, c, p, q, w\}$, and $\{b, c, r, w\}$.

Since there is a minimal model that does not contain w , w is not *true*.

To insert w into **DB** in a minimal way, the algorithm of [31] adds the fact c to the $\mathbf{E}_{\mathbf{DB}}$ (and also removes the fact $(a \vee c)$, which is subsumed by the new fact). This yields two minimal models over the new $\mathbf{E}_{\mathbf{DB}}$: $\{a, c\}$ and $\{b, c\}$. With the $\mathbf{I}_{\mathbf{DB}}$ considered, the database has four minimal models: $\{a, c, p, w\}$, $\{a, c, q, r, w\}$, $\{b, c, p, q, w\}$, and $\{b, c, r, w\}$.

As w is in all four minimal models, it is *true* with respect to the modified database.

Suppose now that **DB** also contains the IC: $\Leftarrow a, c$. The insertion from above cannot work, because a minimal model of the $\mathbf{E}_{\mathbf{DB}}$ contains both a and c . The algorithm takes the ICs into account. In this case, the update is to insert b and c into the $\mathbf{E}_{\mathbf{DB}}$, and to remove $(a \vee b)$ and $(a \vee c)$, which are now subsumed. Note that this would not be a minimal modification without the presence of the IC. After this update, there is only one minimal model projecting over the new $\mathbf{E}_{\mathbf{DB}}$: $\{b, c\}$. There are two minimal models for the new **DB**: $\{b, c, p, q, w\}$ and $\{b, c, r, w\}$.

5.5 Additional Applications

We discuss briefly several other applications of integrity constraints.

1. *Order Optimization.* Many SQL operations such as **group by**, **order by**, **distinct** require the physical ordering of data. Sorting is an expensive operation, however. [85] describe an optimization technique for minimizing the number of sorting columns or detecting when sorting can be avoided altogether because of the existence of functional dependencies on retrieved data. For example, if tuples need to be sorted on columns A and B and there is a functional dependency $A \rightarrow B$, then sorting on B can be avoided. Functional dependencies can also be used to push sort down into a query tree to make it cheaper or to combine two or more sorts into a single sort.

2. *Enhancing the applications of query folding.* Query folding is a semantic optimization technique that replaces the original predicates of a query with so-called resource predicates and answers the modified query. Resources can be views or cached results of previous queries. Query folding can be used for evaluating queries in the context of heterogeneous database systems where tables in different databases are related through view definitions or in semantic query caching where the query may be answered from a cache. Integrity constraints can dramatically increase the applicability of query folding; [27] describes a query folding algorithm that uses functional dependencies; [54] describes an algorithm that uses inclusion dependencies.
3. *Revealing the semantics of object-oriented databases.* One of the reasons for the rapid development of object-oriented databases was their model, which was designed to capture more meaning of an application environment than was possible with a model of relational or even deductive databases. The introduction of a new, more natural object model required the introduction of a new data and query language compatible with that model. This meant, in particular, that many of the semantic-based techniques¹⁵ designed for relational and deductive databases were not directly applicable to the object model; such techniques had to be developed specifically for the new model ([91] and [9]).

It has been shown [50], however, that the existing techniques can be transferred from the relational to the object database model. The main idea of this approach is the transformation of an object schema into a relational schema in such a way that the object semantics is reflected in integrity constraints at the relational level. The transformation is intended to capture all the semantic information encoded in the object model, such as object identity, inheritance, types of relationships between classes (one-to-one, many-to-one) and keys. Consider the following example.

Example 4. Let C_1 be a subclass of C_2 in the object model. Then, C_1 and C_2 can be represented as tables T_1 and T_2 so that each tuple in T_1 and T_2 represents an object from the classes C_1 and C_2 respectively. Then, the fact that C_1 is a subclass of C_2 can be expressed by the following integrity constraint:

$$t_2(X_1, \dots, X_n) \Leftarrow t_1(X_1, \dots, X_n, X_{n+1}, \dots, X_{n+m})$$

Once the transformation of an object schema into a relational schema with integrity constraints is completed, each query posed to an object database can be translated to a query on the relational database. At this point many of the existing tools developed for relational databases can be applied to the query and the result translated back into the object model.

4. *Semantic Data Security via Security Constraints.* Security and privacy are important issues in database systems. All commercial relational systems provide security mechanisms that allow one to designate certain tables to be viewable or updatable by certain users but not by others. Most provide an account and password facility to authenticate the users of the system. However, the granularity of these mechanisms is coarse: permission is often decided on a table-by-table basis.

A finer mode of data security would be useful. One could specify the *types* of data that a given user or class is not permitted to see. This is sometimes called *semantic data security*. The question then is how these semantic security specifications can be accomplished. In [87], it is argued that security constraints should be viewed as a special form of integrity constraints. This would allow security constraints to be written in a declarative fashion, in the same way ICs are. This is akin to user constraints, mentioned in Section 3.3. The difference is that instead of shielding certain answers to the user's query because the user has no interest in them (indicated by conflicts with the user constraints), they are shielded from the user for security or privacy reasons. In [45], the same mechanisms used in semantic query optimization are used also to eliminate answers that would conflict with such security constraints, thereby abiding semantic data security.

¹⁵Such techniques include *semantic query optimization* (see Section 5.1), *semantic query caching* ([23], [15], and [57]), *cooperative answering* (see Section 5.2) and *query rewriting using views* ([81], [65], [14], and [90]).

The issues of semantic data security can be epistemically more complex, if one also wants to ensure that a user can never infer a “secret”, and thereby compromise database security. In [5], first-order formulas are allowed to be designated as *secrets* within a deductive database domain. The deductive system is extended to ensure secrets. It is shown that if one wants to ensure that secrets themselves cannot be inferred, sometimes the database system must lie to the user.

Semantic data security is an issue of growing importance. As database applications become more sophisticated, the possibilities that traditional security mechanisms may be circumvented grow. In [76], it is shown how new techniques for knowledge discovery in databases can compromise data security. Pro-active mechanisms, such as the ones security constraints might be able to provide, could help guarantee that security is not breached.

6 Conclusion and Future Directions

We have discussed various aspects of the use of integrity constraints in databases. We have noted that integrity constraints are important not only for the traditional applications of updates and maintaining the consistency of a database, but also as a way to provide semantic information. Using logic to represent integrity constraints—as well as using logic to represent facts, rules, and queries—and to represent user and other constraints provides a uniform representation language that permits all of the tools and techniques of logic to be applied to a database. It permits systems to be developed that are *complete* and *sound*; that is, all answers are found, and all answers are correct with respect to the semantics of the database. We have also stressed that integrity constraints provide a basis for performing many applications such as *semantic query optimization*, providing *cooperative answers*, *combining databases*, handling *inconsistent databases*, and contributing to the solving of the *view update* problem in databases and disjunctive databases. We have also noted that there is no one uniform definition of the semantics of an integrity constraint. We noted that there is the consistency definition, which states that the database must be consistent with the integrity constraints, and the entailment definition, which states that the integrity constraints must be entailed (or be provable) from the database, and other definitions as well. Each definition is relevant and is useful in different contexts, as we have demonstrated in this paper.

Many areas of the use of integrity constraints still need investigating. Such topics include:

- implementation of semantic query optimization;
- temporal databases;
- transactions and updates;¹⁶
- active databases;
- data mining and inductive inference;
- logical foundations of object-oriented deductive databases;
- combining databases;
- multimedia databases; and
- constraint databases and constraint logic programming.

More discussion on each of these topics can be found in [74]. See also [19] on temporal databases and [62] on active databases.

From the above, it is clear that the application of integrity constraints to databases remains a fertile area to explore. Many topics must be investigated to yield intelligent systems beyond what is currently available.

¹⁶See [6] on this topic.

Acknowledgement

We thank the editors and the referees for their constructive comments.

References

- [1] S. Abiteboul, R. Hall, and V. Vianu. *Foundations of Databases*. Addison-Wesley, Reading, Massachusetts, 1995.
- [2] C. Baral, S. Kraus, and J. Minker. Combining multiple knowledge bases. *IEEE Transactions on Knowledge and Data Engineering*, 3(2):208–220, July 1991.
- [3] C. Baral, S. Kraus, J. Minker, and V. S. Subrahmanian. Combining knowledge bases consisting of first order theories. *Computational Intelligence*, 8:45–71, 1992.
- [4] C. Baral, S. Kraus, J. Minker, and V. S. Subrahmanian. Combining default logic databases. *Intl. Journal of Intelligent and Cooperative Info. Systems*, 3(3):319–348, 1994.
- [5] P. A. Bonatti, S. Kraus, and V. S. Subrahmanian. Declarative foundations of secure deductive databases. In J. Biskup and R. Hull, editors, *Fourth International Conference on Database Theory (ICDT'92)*, Lecture Notes in Computer Science, Vol. 646, pages 391–406, Berlin, Oct. 1992. Springer.
- [6] A. Bonner and M. Kifer. Logic Programming for Database Transactions. In Chomicki and Saake [18], chapter 5.
- [7] G. S. Boolos and R. C. Jeffrey. *Computability and Logic*. Open University Set Book. Cambridge University Press, third edition, 1989.
- [8] F. Bry. A compositional semantics for logic programs and deductive databases. In *Proceedings of the Joint International Conference and Symposium on Logic Programming*, Bad Honnef, Germany, Sept. 1996. MIT Press.
- [9] M. Buchheit, M. A. Jeusfeld, W. Nutt, and M. Staudt. Subsumption between queries to object-oriented databases. *Information Systems*, 19(1):33–54, Jan. 1994.
- [10] U. S. Chakravarthy, J. Grant, and J. Minker. Foundations of semantic query optimization for deductive databases. In J. Minker, editor, *Proceedings of the Workshop on Foundations of Deductive Databases and Logic Programming*, pages 67–101, Washington, D.C., Aug. 1986.
- [11] U. S. Chakravarthy, J. Grant, and J. Minker. Semantic query optimization: Additional constraints and control strategies. In L. Kerschberg, editor, *Proceedings of Expert Database Systems*, pages 259–269, Charleston, Apr. 1986.
- [12] U. S. Chakravarthy, J. Grant, and J. Minker. Logic based approach to semantic query optimization. *ACM Transactions on Database Systems*, 15(2):162–207, June 1990.
- [13] E. Chan. A possible world semantics for disjunctive databases. *IEEE Transactions on Data and Knowledge Engineering*, 5(2):282–292, 1993.
- [14] S. Chaudhuri, R. Krishnamurthy, S. Potamianos, and K. Shim. Optimizing queries with materialized views. In *Proceedings of the 11th ICDE*, pages 190–200, 1995.
- [15] C. M. Chen and N. Roussopoulos. The implementation and performance evaluation of the ADMS query optimizer: Integrating query result caching and matching. In *Proceedings of the 4th International Conference on Extending Database Technology*, Cambridge, U.K., 1994.
- [16] L. Cholvy. Answering queries addressed to a rule base. *Revue d'intelligence artificielle*, 4(1):79–98, 1990.
- [17] L. Cholvy and R. Demolombe. Querying a rule base. In L. Kerschberg, editor, *Expert Database Systems*, Tysons Corner, Virginia, 1987.

- [18] J. Chomicki and G. Saake, editors. *Logics for Databases and Information Systems*. Kluwer Academic Publishers, Boston, 1997.
- [19] J. Chomicki and D. Toman. Temporal Logic in Information Systems. In Chomicki and Saake [18], chapter 3.
- [20] K. L. Clark. Negation as Failure. In H. Gallaire and J. Minker, editors, *Logic and Data Bases*, pages 293–322. Plenum Press, New York, 1978.
- [21] S. Conrad. A Logic Primer. In Chomicki and Saake [18], chapter 2.
- [22] V. Dahl and P. Saint-Dizier, editors. *Natural Language Understanding and Logic Programming*. North Holland, 1988.
- [23] S. Dar, M. Franklin, B. Jónsson, D. Srivastava, and M. Tan. Semantic data caching and replacement. In *Proceedings of VLDB*, 1996.
- [24] S. K. Das. *Deductive Databases and Logic Programming*. Addison-Wesley, Wokingham, England, 1992.
- [25] R. Demolombe and T. Imielinski, editors. *Nonstandard Queries and Nonstandard Answers*. Studies in Logic and Computation 3. Clarendon Press, Oxford, 1994.
- [26] R. Demolombe and A. Jones. Integrity constraints revisited. *Journal of IGPL*, 4(3):369–383, 1996.
- [27] O. Duschka and A. Levy. Recursive plans for information gathering. In *Proc. of 15th IJCAI*, Nagoya, Japan, Aug. 1997.
- [28] T. Eiter, G. Gottlob, and H. Mannila. Disjunctive datalog. *ACM Transactions on Database Systems*, 22(3):364–418, 1997.
- [29] F. Fages. A new fixpoint semantics for general logic programs compared with the well-founded and the stable model semantics. *New Generation Computing*, 9:425–443, 1991.
- [30] J. A. Fernández. *Disjunctive Deductive Databases*. PhD thesis, University of Maryland, Department of Computer Science, College Park, 1994.
- [31] J. A. Fernández, J. Grant, and J. Minker. Model theoretic approach to view updates in deductive databases. *Journal of Automated Reasoning*, 17(2):171–197, 1996.
- [32] J. A. Fernández, J. Lobo, J. Minker, and V. Subrahmanian. Disjunctive LP + integrity constraints = stable model semantics. *Annals of Mathematics and Artificial Intelligence*, 8(3–4):449–474, 1993.
- [33] T. Gaasterland. *Cooperative Answers for Database Queries*. PhD thesis, University of Maryland, Department of Computer Science, College Park, 1992.
- [34] T. Gaasterland, P. Godfrey, and J. Minker. An overview of cooperative answering. *Journal of Intelligent Information Systems*, 1(2):123–157, 1992. Invited paper.
- [35] T. Gaasterland, P. Godfrey, and J. Minker. An overview of cooperative answering. In Demolombe and Imielinski [25], chapter 1, pages 1–40. Appears originally as [34].
- [36] T. Gaasterland and J. Lobo. Qualified answers that reflect user needs and preferences. In *Proceedings of VLDB*, pages 309–320, Santiago de Chile, Chile, 1994.
- [37] T. Gaasterland and J. Minker. User needs and language generation issues in a cooperative answering system. In P. Saint-Dizier, editor, *ICLP'91 Workshop: Advanced Logic Programming Tools and Formalisms for Language Processing*, pages 1–14, INRIA, Paris, France, June 1991.
- [38] A. Gal. *Cooperative Responses in Deductive Databases*. PhD thesis, Department of Computer Science, University of Maryland, College Park, Maryland, 1988.
- [39] A. Gal and J. Minker. A natural language database interface that provides cooperative answers. *Proceedings of the Second Conference on Artificial Intelligence Applications*, December 11–13 1985.
- [40] A. Gal and J. Minker. Informative and cooperative answers in databases using integrity constraints. In Dahl and Saint-Dizier [22], pages 277–300.
- [41] A. V. Gelder, K. Ross, and J. S. Schlipf. Unfounded sets and well-founded semantics for general logic programs. In *Proceedings of the 7th Symposium on Principles of Database Systems*, pages 221–230, 1988.

- [42] M. Gelfond and V. Lifschitz. The stable model semantics for logic programming. In R. A. Kowalski and K. A. Bowen, editors, *Proc. 5th International Conference and Symposium on Logic Programming*, pages 1070–1080, Seattle, Washington, August 15–19 1988.
- [43] P. Godfrey. *An Architecture and Implementation for a Cooperative Database System*. PhD thesis, University of Maryland at College Park, College Park, Maryland 20742, 1997. In progress.
- [44] P. Godfrey. Minimization in cooperative response to failing database queries. *International Journal of Cooperative Information Systems (IJCIS)*, 6(2):95–149, June 1997.
- [45] P. Godfrey and J. Gryz. A framework for intensional query optimization. In D. Boulanger, U. Geske, F. Giannotti, and D. Seipel, editors, *Proceedings of the Workshop on Deductive Databases and Logic Programming*, GMD-Studien Nr. 295, pages 57–68, Bonn, Germany, Sept. 1996. GMD-Forschungszentrum. Held in conjunction with IJCSLP'96.
- [46] P. Godfrey and J. Gryz. Intensional query optimization. Technical Report CS-TR-3702, UMIACS-TR-96-72, Dept. of Computer Science, University of Maryland, College Park, MD 20742, Oct. 1996.
- [47] P. Godfrey and J. Gryz. Overview of dynamic query evaluation in intensional query optimization. In *Proceedings of the Fifth International Conference on Deductive and Object-Oriented Databases (DOOD)*, Montreux, Switzerland, Dec. 1997. See also [46].
- [48] P. Godfrey, J. Gryz, and J. Minker. Semantic query optimization for bottom-up evaluation. In Z. W. Raś and M. Michalewicz, editors, *Foundations of Intelligent Systems: Proceedings of the 9th International Symposium on Methodologies for Intelligent Systems*, Lecture Notes in Artificial Intelligence 1079, pages 561–571, Berlin, June 1996. Springer.
- [49] P. Godfrey, J. Minker, and L. Novik. An architecture for a cooperative database system. In W. Litwin and T. Risch, editors, *Proceedings of the First International Conference on Applications of Databases*, Lecture Notes in Computer Science 819, pages 3–24. Springer Verlag, Vadstena, Sweden, June 1994.
- [50] J. Grant, J. Gryz, J. Minker, and L. Raschid. Semantic query optimization in object databases. In *Proceedings of ICDE*, pages 444–453, Birmingham, UK, Apr 7–11 1997. IEEE Computer Science Press, Los Alamitos, California.
- [51] J. Grant, J. Horty, J. Lobo, and J. Minker. View updates in stratified disjunctive databases. *Journal Automated Reasoning*, 11:249–267, March 1993.
- [52] J. Grant and J. Minker. Integrity constraints in knowledge based systems. In *Knowledge Engineering*, volume II, pages 1–25. McGraw-Hill, New York, 1990.
- [53] P. W. P. J. Grefen and P. M. G. Apers. Integrity control in relational database systems: An overview. *Data and Knowledge Engineering*, 10:187–223, 1993.
- [54] J. Gryz. Query folding with inclusion dependencies. In *Proc. of 14th ICDE*, Orlando, Florida, 1998.
- [55] M. Hammer and S. Zdonik. Knowledge-based query processing. *Proc. 6th International Conference on Very Large Data Bases*, pages 137–147, Oct. 1980.
- [56] T. Imieliński. Intelligent query answering in rule based systems. In Minker [73].
- [57] A. M. Keller and J. Basu. A predicate-based caching scheme for client-server database architectures. *The VLDB Journal*, 5(2):35–47, Apr. 1996.
- [58] J. King. QUIST: A system for semantic query optimization in relational databases. *Proc. 7th International Conference on Very Large Data Bases*, pages 510–517, Sept. 1981.
- [59] R. Kowalski. Logic for data description. In H. G. J. Minker, editor, *Logic and Data Bases*, pages 77–102. Plenum Press, New York, 1978.
- [60] R. Kowalski and F. Sadri. Knowledge representation without integrity constraints, Dec. 1988. Draft manuscript.
- [61] L. V. S. Lakshmanan and R. Missaoui. On semantic query optimization in deductive databases. In *Proc. IEEE International Conference on Data Engineering*, pages 368–375, 1992.

- [62] G. Lausen, B. Ludäscher, and May. On Logical Foundations of Active Databases. In Chomicki and Saake [18], chapter 12.
- [63] S. Lee and J. Han. Semantic query optimization in recursive databases. In *Proc. IEEE International Conference on Data Engineering*, pages 444–451, 1988.
- [64] S. Lee, L.J.Henschen, and G. Qadah. Semantic query reformulation in deductive databases. In *Proc. IEEE International Conference on Data Engineering*, pages 232–239. IEEE Computer Society Press, 1991.
- [65] A. Y. Levy, A. O. Mendelzon, Y. Sagiv, and D. Srivastava. Answering queries using views. In *Proc. PODS*, pages 95–104, 1995.
- [66] A. Y. Levy and Y. Sagiv. Semantic query optimization in datalog programs. In *Proceedings of the ACM Symposium on the Principles of Database Systems (PODS)*, 1995.
- [67] J. W. Lloyd. *Foundations of Logic Programming*. Symbolic Computation—Artificial Intelligence. Springer-Verlag, Berlin, second edition, 1987.
- [68] J. Lobo, J. Minker, and A. Rajasekar. *Foundations of Disjunctive Logic Programming*. M.I.T. Press, Cambridge, Massachusetts, 1992.
- [69] D. Loveland. Theorem-provers combining model elimination and resolution. In Meltzer and Michie, editors, *Machine Intelligence 4*, pages 73–86. University Press, Edinburgh, 1969.
- [70] J. McSkimin. *The Use of Semantic Information in Deductive Question-Answering Systems*. PhD thesis, University of Maryland, College Park, Maryland 20742, 1976.
- [71] J. Melton and A. R. Simon. *Understanding the New SQL: A Complete Guide*. Morgan Kaufmann, San Mateo, California, 1993.
- [72] J. Minker. On indefinite databases and the closed world assumption. In *Proceedings of the Sixth Conference on Automated Deduction*, pages 292–308, 1982. Also in: *Lecture Notes in Computer Science* 138, pages 292–308. Springer Verlag, 1982.
- [73] J. Minker, editor. *Foundations of Deductive Databases and Logic Programming*. Morgan Kaufmann Pub., 1988.
- [74] J. Minker. Logic and databases: a 20 year retrospective. In *Workshop on Logic in Databases*, San Miniato, Italy, July 1996. Invited Keynote Address.
- [75] A. Motro. Using constraints to provide intensional answers to relational queries. In *Proceedings of the Fifteenth International Conference on Very Large Data Bases*, Aug. 1989.
- [76] D. E. O’Leary. Knowledge discovery as a threat to database security. In Piatetsky-Shapiro and Frawley [77], chapter 30.
- [77] G. Piatetsky-Shapiro and W. J. Frawley, editors. *Knowledge Discovery in Databases*. AAAI Press and MIT Press, Menlo Park, California, 1991.
- [78] A. Pirotte and D. Roelants. Constraints for improving the generation of intensional answers in deductive databases. In *Proceedings of the 5th IEEE International Conference on Data Engineering*, 1989.
- [79] A. Pirotte, D. Roelants, and E. Zimanyi. Controlled generation of intensional answers. *IEEE Transactions on Knowledge and Data Engineering*, 1990.
- [80] S. Pradhan and J. Minker. Using priorities to combine knowledge bases. *Journal of Cooperative Information Systems*, 5(2,3):333–364, Feb. 1997.
- [81] X. Qian. Query folding. In *Proceedings of the 12th International Conference on Data Engineering*, pages 48–55, 1996.
- [82] R. Reiter. On Closed World Data Bases. In H. Gallaire and J. Minker, editors, *Logic and Data Bases*, pages 55–76. Plenum Press, New York, 1978.
- [83] R. Reiter. What should a database know? *Journal of Logic Programming*, 14(1& 2):127–153, Oct. 1992.
- [84] C. Sakama. Possible model semantics for disjunctive databases. In *Proceedings of the First International Conference on Deductive and Object Oriented Databases (DOOD’89)*, pages 337–351, Kyōto, Dec. 1989.

- [85] D. Simmen, E. Shekita, and T. Malkems. Fundamental techniques for order optimization. In *Proceedings of SIGMOD*, pages 57–67, 1996.
- [86] M. Stickel. A PROLOG technology theorem prover: Implementation by an extended PROLOG compiler. *Journal of Automated Reasoning*, 4(4):353–380, 1988.
- [87] B. Thuraisingham and W. Ford. Security constraint processing in a multilevel secure distributed database management system. *IEEE Transactions on Knowledge and Data Engineering*, 7(2):274–293, Apr. 1995.
- [88] J. D. Ullman. *Principles of Database and Knowledge-Base Systems, Volumes I & II*. Principles of Computer Science Series. Computer Science Press, Incorporated, Rockville, Maryland, 1988/1989.
- [89] R. van der Meyden. Logical Approaches to Incomplete Information: A Survey. In Chomicki and Saake [18], chapter 10.
- [90] H. Z. Yang and P.-Å. Larson. Query transformation for PSJ-queries. In *Proceedings of the Thirteenth International Conference on Very Large Data Bases*, pages 245–254, 1987.
- [91] J. Yoon and L. Kerschberg. Semantic query optimization in deductive object-oriented databases. In *Proceedings of the 3rd International Conference on Deductive and Object-Oriented Databases*, pages 169–182, 1993.

Index

- aggregation, 11
- clause, 5–7, 9, 11–13, 15–19
 - definite, 6
 - ground, 17
 - null, 17, 18
- closed world assumption (*CWA*), 6, 27
- completion, 6
- cooperative answering, 21–24
- database
 - deductive, 3, 6–8, 13, 14, 24, 26, 28, 29
 - definite, 6, 13, 16
 - disjunctive, 6, 13, 15–17, 24, 26, 27
 - distributed, 24
 - extensional, 26
 - extensional (**E_{DB}**), 6, 8
 - heterogeneous, 24, 28
 - intensional (**I_{DB}**), 6, 8
 - logic, 3, 5–7, 11
 - multimedia, 30
 - normal, 7, 8, 10, 14, 16
 - object-oriented, 4, 21, 28
 - object-relational, 4
 - relational, 3, 4, 6, 8, 21, 26, 28
 - stable, 8
 - stratified, 7, 25, 26
- Datalog, 3, 6, 7, 11, 13, 24
- deduction, 3, 4
- false presupposition, 23
- formula
 - atomic, 5
 - clausal, *see* clause
 - closed, 11
 - function-free, 5
 - ground, 5
 - query, *see* query
- integrity constraint, 3–30
 - denial, 10–13, 15, 17, 27
 - domain, 9
 - functional dependency, 3, 9, 22, 28
 - inclusion dependency, 4, 10, 28
 - preference, 14
 - referential, 10
 - security, 28, 29
 - state, 14, 24
 - static, 14
 - temporal, 14
 - universal, 14
 - user, 14, 22, 29
- knowledge discovery, 23
- logic
 - default, 25
 - first-order, 3, 5, 7, 8, 11, 25
 - higher order, 5
 - modal, 8, 14
 - non-classical, 4
 - non-monotonic, 7, 11, 27
 - temporal, 8
- misconception, 21–24
- model, 16
 - canonical, 16
 - Herbrand, 11
 - minimal, 11–13, 15–17, 24, 27
 - perfect, 7, 27
 - stable, 13, 16, 27
 - unique, 8, 12
- negation, 6–8
 - default, 7, 10, 11, 13, 15, 16
 - logical, 7, 13
- Prolog, 9
- query, 5, 6, 22
 - answer to, 5, 21, 22, 24
 - empty, 21, 24
 - intensional, 22
 - caching, 28
 - closed, 3
 - folding, 28
 - optimization, 27
 - semantic, 19–21, 24, 29
- recursion, 6, 7, 20
- residue, 15–20, 22
- rule, 3–6, 8, 9, 11, 12, 14–17, 20, 26
 - body of
 - empty, 6
 - deductive, 14
 - definite, 6, 15
 - disjunctive, 6, 7, 27
 - head of, 5
 - empty, 6
 - Horn, 6
 - normal, 7
 - range-restricted, 6, 10

- semantics, 3, 7–14, 16, 21, 22, 24, 28
 - Herbrand, 27
 - model, 8, 9, 11–13
 - stable, 8
 - well-supported, 8
 - well-founded, 8
- SLDNF refutation, 15
- SLI refutation, 17
- substitution, 5, 17
 - ground, 5
 - inverse, 5
- subsumption, 17, 22
 - partial, 17, 18, 22
- view, 3–6
 - update, 16, 26–27